



Movilidad accesible para todos

Autor: Jorge Ferrer Gil

Ciclo: GS Desarrollo de Aplicaciones Multiplataforma (DAM)

Curso: 2025 – 2026

Índice

1.	Introducción.....	3
1.1.	Descripción del proyecto	
1.2.	Objetivos	
1.3.	Planificación	
1.4.	Presupuesto	
2.	Análisis de mercado.....	6
2.1.	Público objetivo y perfiles de usuario	
2.2.	Análisis de competencia	
2.3.	Propuesta de valor	
2.4.	Funcionalidades del producto	
3.	Diseño.....	8
3.1.	Diagrama de navegación	
3.2.	Wireframes (Baja fidelidad)	
3.3.	Interfaces finales (Fidelidad alta)	
4.	Desarrollo.....	11
4.1.	Arquitectura del sistema	
4.2.	Herramientas utilizadas	
4.3.	Lenguaje, frameworks y librerías	
4.4.	Modelo de datos	
4.5.	Instrucciones de despliegue	
5.	Conclusión y líneas de futuro.....	17

INTRODUCCIÓN

1.1 Descripción del proyecto

MOBII es una aplicación Android pensada para ayudar a personas con discapacidad a moverse de forma más autónoma por la ciudad de Valencia. La idea surgió de un problema concreto: las personas con movilidad reducida u otras discapacidades tienen muchas dificultades para encontrar aparcamientos adaptados, reservar transporte accesible o localizar centros especializados, y toda esa información está dispersa en webs distintas que no siempre funcionan bien desde el móvil.

El proyecto no se ha quedado solo en la app. Se ha construido también un backend propio con Node.js que gestiona la autenticación y almacena los datos de los usuarios, y un panel de administración web con Angular desde el que se pueden gestionar las cuentas registradas. La combinación de las tres capas es lo que hace al proyecto completo desde el punto de vista técnico.

Tres capas independientes pero conectadas: app Android nativa (Java) + API REST propia (Node.js + Express + SQLite) + panel web de administración (Angular 21).

1.2 Objetivos

- Centralizar en una sola app toda la información útil de movilidad accesible en Valencia.
- Permitir a los usuarios registrar su tipo de discapacidad para que el servicio sea más personalizado.
- Ofrecer un sistema de reserva de taxi adaptado directamente desde la aplicación.
- Mostrar aparcamientos accesibles en un mapa interactivo con la opción de marcarlos como favoritos.
- Dar acceso a un directorio de centros especializados según el tipo de discapacidad.
- Proporcionar al administrador una herramienta web para gestionar los usuarios del sistema.

1.3 Planificación

Fase	Descripción	Duración estimada
Análisis y diseño	Definición de requisitos, wireframes y modelo de datos	2 semanas
Backend	API REST, autenticación JWT, endpoints CRUD	2 semanas
App Android	Pantallas, navegación, integración con API y APIs externas	4 semanas
Panel admin	Angular, gestión de usuarios, detalle de perfil	2 semanas
Sincronización	Push sync de favoritos y citas entre app y backend	1 semana
Pruebas y ajustes	Testing manual, corrección de bugs, dark mode	1 semana

1.4 Presupuesto

Concepto	Herramienta / Recurso	Coste
Entorno de desarrollo	Android Studio, VS Code	0 € (gratuitos)
APIs de datos	Open Data Ayuntamiento de Valencia	0 € (públicas)
Mapas	Leaflet.js + OpenStreetMap	0 € (open source)
Servidor de pruebas	Localhost / equipo propio	0 €
Horas de desarrollo (estimado)	~120 horas a 15 €/h	1.800 €
Total		1.800 €

ANÁLISIS DE MERCADO

2.1 Público objetivo y perfiles de usuario

El público principal de MOBII son personas con algún tipo de discapacidad o movilidad reducida que viven o se desplazan habitualmente por Valencia. También puede ser útil para familiares o cuidadores que gestionan los desplazamientos de otras personas.

Usuario final

Persona con discapacidad física, sensorial o intelectual. Utiliza la app para encontrar aparcamientos adaptados, reservar taxis y consultar centros especializados. Necesita una interfaz simple, con buen contraste y opciones de accesibilidad.

Administrador

Gestor del sistema que accede al panel web. Puede ver, crear, editar y eliminar cuentas de usuario. También tiene acceso a los datos de accesibilidad declarados por cada usuario y un contador de su actividad en la app.

2.2 Análisis de la competencia

Antes de ponerse a desarrollar, vale la pena mirar qué hay ya en el mercado que se solape con lo que queremos hacer.

Google Maps

La referencia en navegación. Permite buscar parking pero no filtra específicamente por accesibilidad para personas con discapacidad ni ofrece ningún tipo de reserva de transporte adaptado.

Muy conocida

Mapas precisos

Sin filtro accesibilidad

Sin reservas

Apparcatoo

App de aparcamiento que muestra plazas libres en tiempo real en algunas ciudades. No incluye plazas para personas con movilidad reducida de forma diferenciada ni conecta con ningún sistema de transporte.

Tiempo real

Sin accesibilidad

Solo parking

Cabify Assist

Cabify ofrece una modalidad de vehículo accesible en algunas ciudades, pero es un servicio de pago sin ninguna integración con aparcamientos ni centros especializados. No está disponible en todas las zonas de Valencia.

Transporte real

De pago

Sin info accesibilidad

Web Ayuntamiento Valencia

El ayuntamiento publica datos de accesibilidad en su portal Open Data, pero no existe una app que los consuma y los presente de forma usable. Es precisamente de ahí de donde MOBII extrae los datos de parking y centros.

Datos oficiales

No es app

Difícil de usar

2.3 Propuesta de valor

Después de revisar lo que hay disponible, queda bastante claro que no existe una solución que combine todo lo que necesita una persona con discapacidad en Valencia en un solo lugar. MOBII cubre ese hueco: aparcamientos adaptados, reserva de transporte, directorio de centros por tipo de discapacidad y actividades de ocio accesible, todo en una app con soporte de modo oscuro y perfil de accesibilidad personalizable.

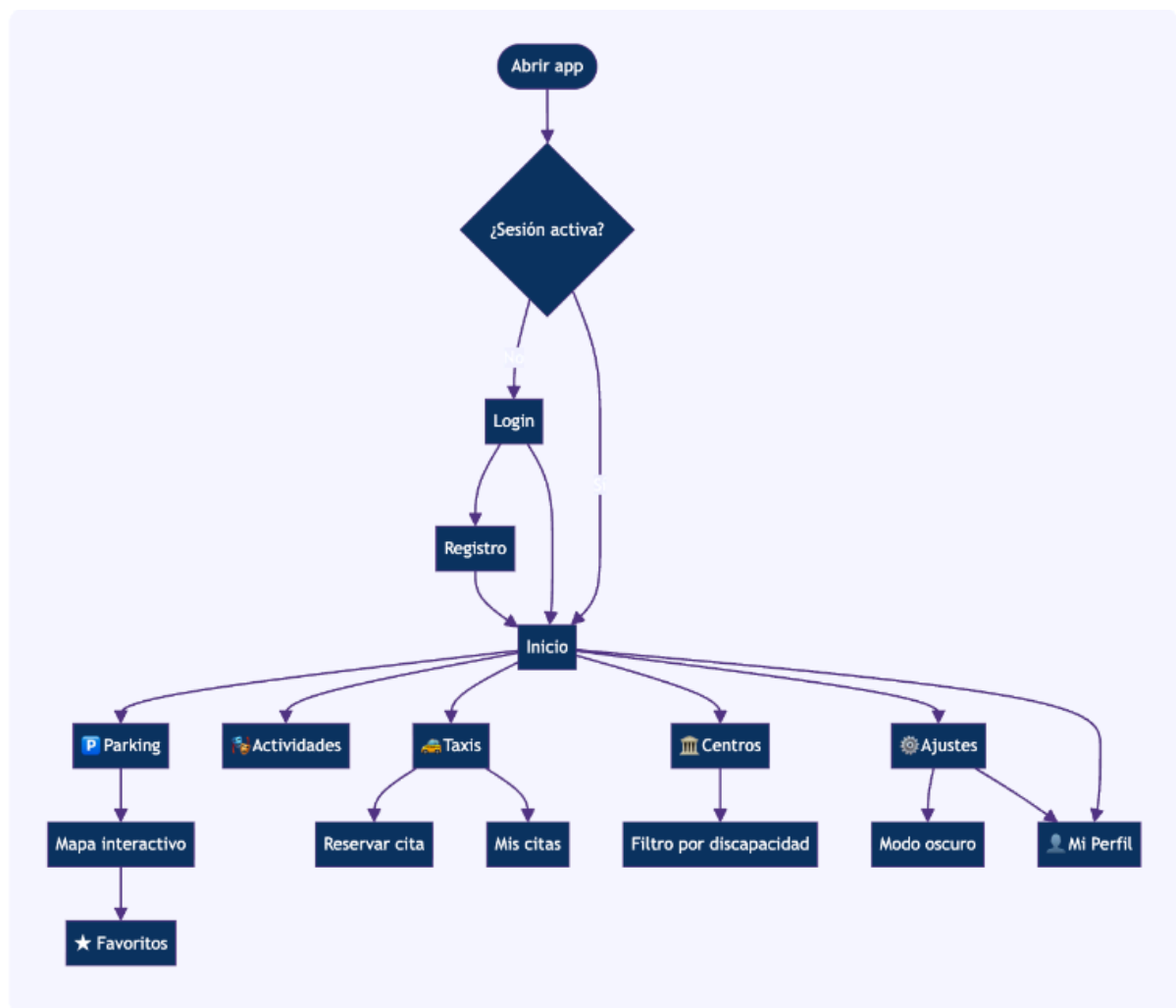
2.4 Funcionalidades del producto

Funcionalidad	Descripción	Capa
Registro e inicio de sesión	Autenticación JWT con token de 7 días. El usuario puede registrarse y el token se almacena localmente.	App + Backend
Perfil de accesibilidad	El usuario puede indicar su tipo de discapacidad. El admin la visualiza desde el panel.	App + Backend
Mapa de aparcamientos	Datos en tiempo real del Open Data de Valencia. El usuario puede marcar favoritos que se guardan local y en la nube.	App + Backend
Reserva de taxi adaptado	Formulario de reserva con dirección de recogida, destino, fecha, hora y plazas. Las citas se sincronizan con el backend.	App + Backend
Centros especializados	Mapa con centros filtrados por tipo de discapacidad (general, física, intelectual, sensorial), datos del Geoportal de Valencia.	App
Actividades de ocio accesible	Listado de actividades con filtro por categoría y posibilidad de guardar favoritas.	App
Modo oscuro	El usuario puede activar el modo oscuro desde ajustes. Funciona en toda la app.	App
Panel de administración	Interfaz web para gestionar usuarios: crear, editar, eliminar, ver discapacidad declarada y contadores de actividad.	Panel web

DISEÑO

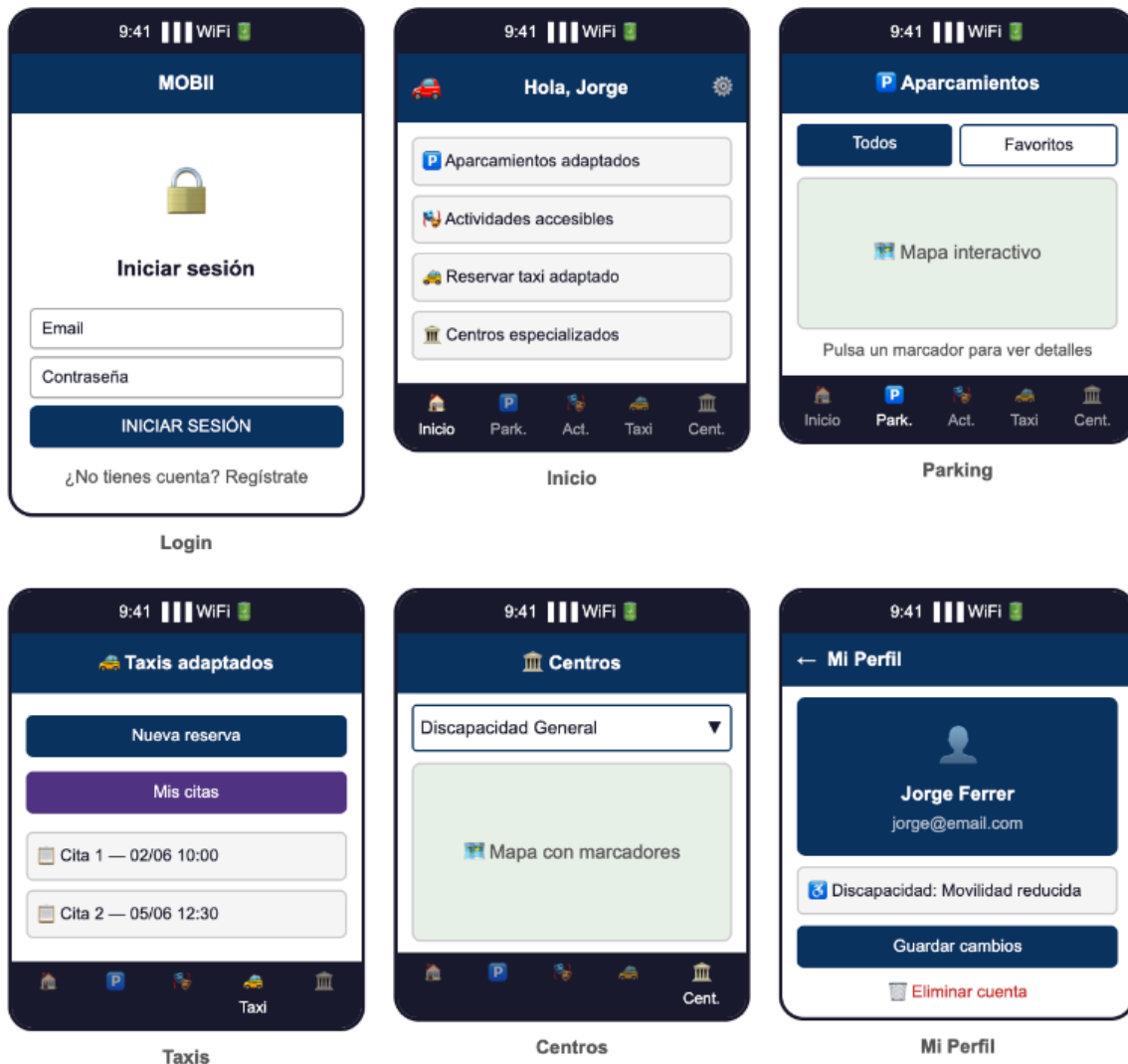
3.1 Diagrama de navegación (User Flow)

El flujo principal de la app arranca en el login. Si el usuario tiene sesión guardada va directamente a la pantalla principal. Desde ahí puede acceder a las cinco secciones del menú inferior. Las únicas pantallas adicionales son Ajustes y Perfil, que se abren desde la barra superior.



3.2 Wireframes — Baja fidelidad

Los primeros bocetos se hicieron a mano para definir la estructura de cada pantalla antes de escribir ninguna línea de código. Lo más importante en esta fase fue decidir el patrón de navegación: se optó por una *bottom navigation* con cinco secciones para que fuera fácil de usar con una sola mano, algo especialmente relevante dado el público objetivo.



3.3 Interfaces finales — Alta fidelidad

La paleta de colores elegida gira en torno a un azul oscuro como color principal (#1A1A2E / #0F3460) que transmite seriedad y confianza, con un morado como acento (#533483). Se eligieron estos colores porque ofrecen un contraste alto con el fondo blanco, algo especialmente importante teniendo en cuenta que parte del público objetivo puede tener discapacidad visual.



La tipografía usa el sistema de la plataforma Android (*Roboto*) sin necesidad de cargar fuentes externas. Los tamaños siguen las recomendaciones de Material Design: 18sp para títulos de sección, 15sp para cuerpo principal y 12sp para texto secundario.

Modo oscuro: la app incluye soporte completo de modo oscuro implementado con `values-night/colors.xml` y `AppCompatDelegate`. En modo oscuro el fondo pasa a #121212 y las superficies a #1E1E1E, manteniendo el mismo contraste que en modo claro.

Panel de administración web

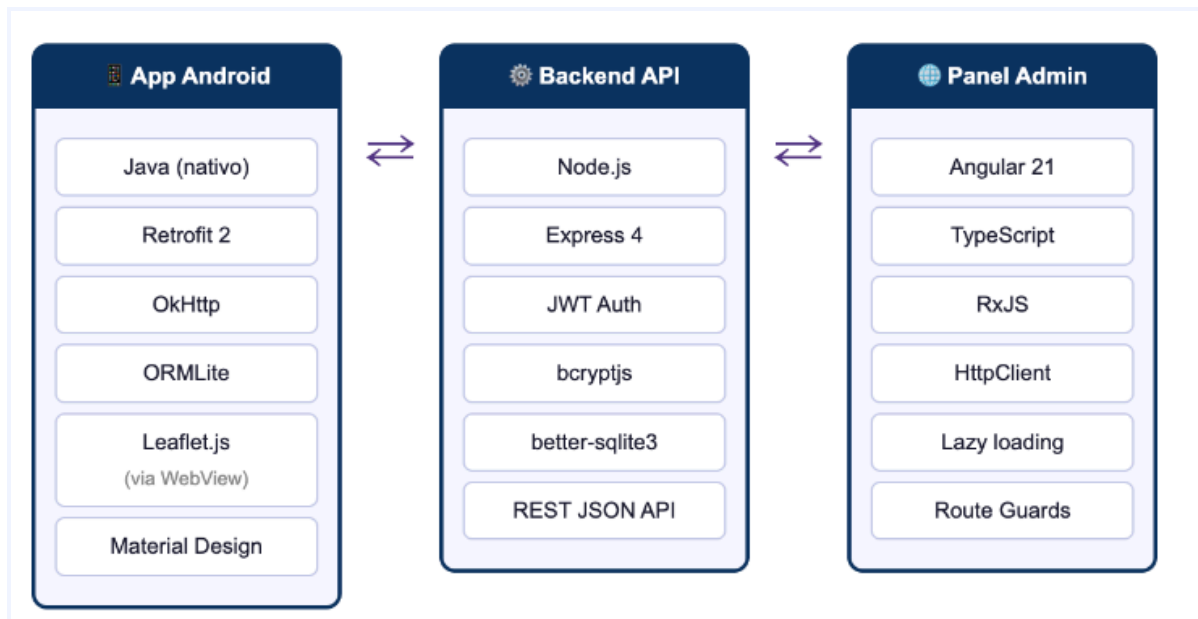
El panel web tiene un sidebar con navegación a la sección de usuarios. Cada fila de la tabla incluye tres acciones: ver detalle, editar y eliminar. Al pulsar "Ver más" aparece un modal con la discapacidad declarada por el usuario y contadores de parking favoritos y citas reservadas sincronizadas desde la app.



DESARROLLO

4.1 Arquitectura general del sistema

El sistema se divide en tres componentes independientes que se comunican entre sí a través de una API REST. Esta separación en capas tiene una ventaja clara: si en el futuro se quisiera cambiar el frontend por una app iOS, por ejemplo, el backend no necesitaría tocar nada.



APIs externas consumidas: la app consulta directamente el **Open Data del Ayuntamiento de Valencia** para obtener los datos de aparcamientos adaptados, y el **Geoportal ArcGIS de Valencia** (capas 17-20 del servicio SociedadBienestar) para los centros especializados por tipo de discapacidad.

4.2 Herramientas de desarrollo

Herramientas principales		Entorno y despliegue	
Android Studio	IDE para el desarrollo de la app Android	Emulador Android	Pixel 6, API 33 (Android 13)
Visual Studio Code	Editor para el backend y el panel Angular	Node.js v24	Entorno de ejecución del backend
Postman	Pruebas de los endpoints de la API REST	npm	Gestión de dependencias JS
Git	Control de versiones del proyecto	Gradle	Build system de la app Android
Chrome DevTools	Depuración del WebView y del panel web	ng CLI	Compilación y serve del panel Angular

4.3 Lenguajes, frameworks y librerías

Capa	Tecnología	Versión	Uso
App Android	Java	11	Lenguaje principal de la app
	Retrofit 2	2.9.0	Llamadas a la API REST propia
	OkHttp	(transitiva)	Llamadas a APIs externas (GeoJSON)
	ORMLite	5.7	Persistencia local con SQLite
	Gson	(transitiva)	Serialización/deserialización JSON
	Material Components	1.11.0	UI (BottomNav, switches, botones)
Backend	Node.js	24	Entorno de ejecución
	Express	4.x	Framework HTTP, routing
	better-sqlite3	9.x	Base de datos relacional
	jsonwebtoken / bcryptjs	—	Autenticación JWT y hash de contraseñas
Panel web	Angular	21	Framework SPA
	TypeScript	5.x	Lenguaje del frontend
	RxJS	7.x	Programación reactiva, forkJoin

4.4 Modelo de datos

El sistema utiliza dos bases de datos SQLite independientes: una en el backend (gestionada por Node.js con better-sqlite3) y otra local en el dispositivo Android (gestionada por ORMLite). Cuando hay conexión, los datos de favoritos y reservas se sincronizan del dispositivo al servidor.

Base de datos del Backend

📁 usuarios		
🔑 id	INTEGER	PK autoincremental
nombre	TEXT	Nombre completo
email	TEXT	Único identificador
password	TEXT	Hashado con bcrypt
rol	TEXT	usuario / administrador
activo	INTEGER	1 activo, 0 inactivo
discapacidad	TEXT	Nulla declarada por el usuario
created_at	TEXT	Fecha de registro

🅑 parking_favoritos		
🔑 id	INTEGER	PK autoincremental
user_email	TEXT	Email del propietario
record_id	TEXT	ID del Open Data Valencia
direccion	TEXT	Dirección del aparcamiento
created_at	TEXT	Fecha de guardado

🚗 reservas		
🔑 id	INTEGER	PK autoincremental
user_email	TEXT	Email del propietario
nombre	TEXT	Nombre del servicio
recogida / destino	TEXT	Direcciones
fecha / hora	TEXT	Fecha y hora de la cita
plazas / telefono	INT/TEXT	Datos adicionales

Base de datos local (ORMLite — dispositivo Android)

ParkingFavorito

Mismos campos que el backend más userId (email local). Se sincroniza al backend al añadir o eliminar.

Reserva

Incluye todos los campos de la reserva más syncId (ID del backend, -1 si no sincronizado). Permite borrar en el servidor al cancelar.

4.5 Instrucciones de despliegue

Backend (Node.js)

```
# 1. Entrar en la carpeta del backend

cd TFG/backend

# 2. Instalar dependencias

npm install

# 3. Arrancar el servidor (puerto 3000)

node server.js

# Usuario admin por defecto:

email: admin@mobii.com    password: admin123
```

Panel de Administración (Angular)

```
# 1. Entrar en la carpeta del panel

cd TFG/admin-panel

# 2. Instalar dependencias

npm install

# 3. Lanzar en modo desarrollo

ng serve

# Abrir en el navegador:

http://localhost:4200
```

App Android

```
# 1. Abrir el proyecto en Android Studio

File → Open → BaseProyecto/JorgeFerrer_MOBII_BACKUP

# 2. La URL del backend está en:

auth/AuthRetrofitClient.java → BASE_URL

# Emulador (localhost):

http://10.0.2.2:3000/

# Dispositivo físico (cambiar por IP local):

http://192.168.x.x:3000/

# 3. Run → Run app (Shift+F10)
```

 **Nota importante:** el backend y el panel deben estar corriendo antes de abrir la app. La app funciona en modo offline para consultar los datos ya cacheados en ORMLite, pero el login requiere conexión.

CONCLUSIONES Y LÍNEAS DE FUTURO

5.1 Conclusiones

Cuando empecé este proyecto tenía claro que quería hacer algo que no fuera solo un CRUD básico. La idea de enfocarlo en la accesibilidad me pareció interesante desde el principio porque es un problema real y porque me obligaba a pensar en el usuario de una forma distinta a lo habitual.

Lo que más me ha aportado del desarrollo ha sido trabajar con tres capas al mismo tiempo y ver cómo se conectan entre sí. Al principio parece mucho, pero en cuanto tienes el backend funcionando y la app Android ya puede hacer login, el resto va tomando forma bastante rápido. Lo más complicado sin duda fue la parte del mapa con Leaflet.js dentro de un WebView: hay que manejar la comunicación JavaScript-Java con la interfaz `@JavascriptInterface`, y al principio cometí errores de CORS que me costaron bastante tiempo hasta entender que la solución era mover todas las peticiones HTTP al lado Java en vez de hacerlas desde el WebView.

También aprendí mucho sobre la importancia de versionar la base de datos local. Cuando añadí el campo `syncId` a la tabla de reservas tuve que gestionar la migración con `onUpgrade` en ORMLite, y eso me enseñó por qué los proyectos reales necesitan un sistema de migraciones desde el primer día.

En cuanto a lo que no he llegado a completar como me hubiera gustado: la sincronización offline es la parte más frágil. Si el usuario añade un favorito sin conexión, ese dato se queda en el dispositivo y nunca llega al backend a no ser que vuelva a interactuar con la app con conexión activa. Para el alcance de este TFG es suficiente, pero en una versión real necesitaría una cola de sincronización pendiente.

En general estoy satisfecho con el resultado. La app funciona, tiene una interfaz coherente con modo oscuro, y el panel de administración permite gestionar los usuarios de forma cómoda. Creo que el conjunto demuestra que he adquirido las competencias del ciclo.

5.2 Líneas de futuro

Mejora	Descripción	Prioridad
HTTPS en producción	Cambiar el tráfico HTTP por HTTPS con certificado TLS y eliminar <code>usesCleartextTraffic</code> .	Alta
EncryptedSharedPreferences	Guardar el token JWT con la librería Jetpack Security para protegerlo en dispositivos rooteados.	Alta
Rate limiting en login	Usar <code>express-rate-limit</code> para limitar intentos fallidos de inicio de sesión y evitar ataques de fuerza bruta.	Alta
Tests unitarios	Añadir tests del backend con Jest y tests del panel Angular con Jasmine. Aunque sea una cobertura mínima.	Media
Cola de sync offline	Implementar <code>WorkManager</code> para reintentar la sincronización de favoritos y reservas cuando se recupere la conexión.	Media
Notificaciones push	Integrar Firebase Cloud Messaging para recordar las citas de taxi al usuario con antelación.	Media
Geolocalización activa	Ordenar los aparcamientos y centros por distancia al usuario usando el permiso <code>ACCESS_FINE_LOCATION</code> .	Baja
Recuperación de contraseña	Flujo de reset de contraseña por email con token temporal.	Baja
Base de datos en producción	Migrar de SQLite a PostgreSQL para soportar múltiples usuarios concurrentes sin riesgo de bloqueos.	Baja

Reflexión final: MOBII nació como un proyecto de fin de ciclo y ha acabado siendo un sistema completo con tres capas funcionales, autenticación real, integración con datos abiertos del Ayuntamiento de Valencia y sincronización entre dispositivo y servidor. Hay cosas que mejorar, como en cualquier proyecto real, pero la base está ahí y es sólida.